

Reviewer Pack — Minimal Topological Entropy of Curves and Frege Proof Depth ...

Entry fce371b89264 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 03:21:05 UTC

Minimal Topological Entropy of Curves and Frege Proof Depth Correlation via Frobenius Theory

Entry ID: fce371b89264 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 03:21:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically, topological entropy) **Field B** (complexity object): Frege Proof Complexity

Statement:

For every d -regular graph G with n vertices, the minimal topological entropy ($h(G)$) of its associated curve is linearly correlated with its Frege proof depth $w(\varphi_G)$, such that $h(G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

Topological entropy measures the complexity of a space's dynamical system, and algebraic curves can be associated with graphs. Frobenius theory provides a link between algebraic geometry and graph invariants. This conjecture aims to exploit these connections to gain insights into proof complexity.

Taxonomy category: topological_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0f8965f7362dadda

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every d -regular graph G with n vertices, if the correlation coefficient between $h(G)$ and $w(\varphi_G)$ is $|\rho| \geq 0.7$ for at least 90% of seeds, then the conjecture is supported. If any seed produces a correlation coefficient $|\rho| < 0.5$ or $h(G) \neq \Theta(w(\varphi_G))$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Topological Entropy algebraic geometry AND Frege proof complexity - Curve entropy in algebraic geometry related to Frege proof depth - Frobenius theory and correlation between topological entropy of curves and Frege proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(b[i], A[i][i])
        for j in range(i-1, -1, -1):
            b[j] -= A[j][i] * x[i]
    return x

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def adjugate_matrix(A):
    n = len(A)
    if n == 1:
        return [[A[0][0]]]
    submatrices = []
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
            submatrices.append(submatrix)
    cofactors = [[(-1)**(i+j) * det(submatrix) for j in range(n)] for i in range(n)]
    return transpose_matrix(cofactors)

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def transpose_matrix(A):
    n = len(A)
    m = len(A[0])
    B = [[0] * n for _ in range(m)]
    for i in range(n):
        for j in range(m):
            B[j][i] = A[i][j]
    return B

def generate_d_regular_graph(d, n):
    if d * n % 2 != 0:
        raise ValueError("d * n must be even")
    G = [[0] * n for _ in range(n)]
    edges = set()
    while len(edges) < d * n // 2:
        u = random.randint(0, n-1)
        v = random.randint(0, n-1)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            G[u][v] = 1
            G[v][u] = 1
            edges.add((u, v))
    return G

def adjacency_matrix_spectral_radius(G):
    n = len(G)
    A = [[G[i][j] for j in range(n)] for i in range(n)]
    eigenvalues = []
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    for _ in range(20): # Power iteration method
        v = [random.random() for _ in range(n)]
        v = [x / sum(v) for x in v]
        Av = matrix_multiply(A, v)
        lambda_ = max(abs(x) for x in Av)

```

```

        eigenvalues.append(lambda_)
    return max(eigenvalues)

def construct_tseitin_formula(G):
    n = len(G)
    clauses = []
    literals = {}
    for i in range(n):
        literals[i] = random.randint(1, 2*n)
    for i in range(n):
        clause = [literals[i]]
        for j in range(i+1, n):
            if G[i][j]:
                clause.append(-literals[j])
            else:
                clause.append(literals[j])
        clauses.append(clause)
    for i in range(n):
        for j in range(i+1, n):
            if G[i][j]:
                clause = [-literals[i], -literals[j]]
                clauses.append(clause)
    return clauses

def dpll_solver(clauses, assignment):
    def search():
        unassigned_vars = [i for i in range(1, 2*n+1) if i not in assignment]
        if not unassigned_vars:
            return all([eval_clause(c, assignment) for c in clauses])
        var = random.choice(unassigned_vars)
        assignment[var] = True
        if search():
            return True
        del assignment[var]
        assignment[var] = False
        if search():
            return True
        del assignment[var]
        return False

    def eval_clause(clause, assignment):
        for literal in clause:
            var = abs(literal)
            sign = 1 if literal > 0 else -1
            if var in assignment and assignment[var] == (sign == 1):
                return True
        return False

    n = len(assignment) // 2
    return search()

def run_trial(seed: int) -> dict:
    random.seed(seed)
    d = 3
    n_values = [5, 10, 15, 20, 30, 40]
    h_values = []

```

```

w_values = []
instances_tested = 0
n_max = 0

for n in n_values:
    if n > n_max:
        n_max = n
    G = generate_d_regular_graph(d, n)
    h = adjacency_matrix_spectral_radius(G)
    h_values.append(h)
    instances_tested += 1

    phi_G = construct_tseitin_formula(G)
    w = sum(dpll_solver(phi_G, {})) for _ in range(30)) / 30
    w_values.append(w)
    instances_tested += 30

mean_h = sum(h_values) / len(h_values)
mean_w = sum(w_values) / len(w_values)
correlation = sum((h_values[i] - mean_h) * (w_values[i] - mean_w) for i in range(instances_tested)) / (instances_tested * (max(h_values) - min(h_values)) * (max(w_values) - min(w_values)))
conjecture_holds = abs(correlation) >= 0.7
counterexample = "" if conjecture_holds else "correlation_coefficient"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_19f42235.py", line 196, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_19f42235.py", line 166, in run_trial
    G = generate_d_regular_graph(d, n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_19f42235.py", line 81, in generate_d_re
    raise ValueError("d * n must be even")
```

ValueError: d * n must be even

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a ValueError indicating that the product of d and n must be even, which prevented it from running to completion. As a result, no data was produced to evaluate the conjecture. | next: Ensure that the test code correctly handles cases where the product of d and n is not even before rerunning the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21239
2	preregistration	ollama_remote	glm4:latest	0	0	9603
3	novelty	ollama_remote	glm4:latest	0	0	8332
4	novelty	ollama_remote	glm4:latest	0	0	15302
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20265
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38614
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32693
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28172
9	judge	ollama_remote	glm4:latest	0	0	8954

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 183174 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/fce371b89264.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fce371b89264.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fce371b89264.tar.gz` (if generated)